Large Language Models in 2026 and beyond

Before starting.

- I teach 3 level 3/4 modules
  - COMP3074 – Human-AI Interaction
  - COMP4030 – Data Science and Machine Learning
  - COMP3004/4105 – Designing Intelligent Agents
- ML, NLP, HAI, BCI (soon recruiting an intern for this)

Dr Jeremie Clos
jeremie.clos@nottingham.ac.uk
For UG students looking for projects (HAI, ML, NLP):
https://jclos.ovh/project-ideas/

1. NLP systems

↓

2. From statistical to neural language models

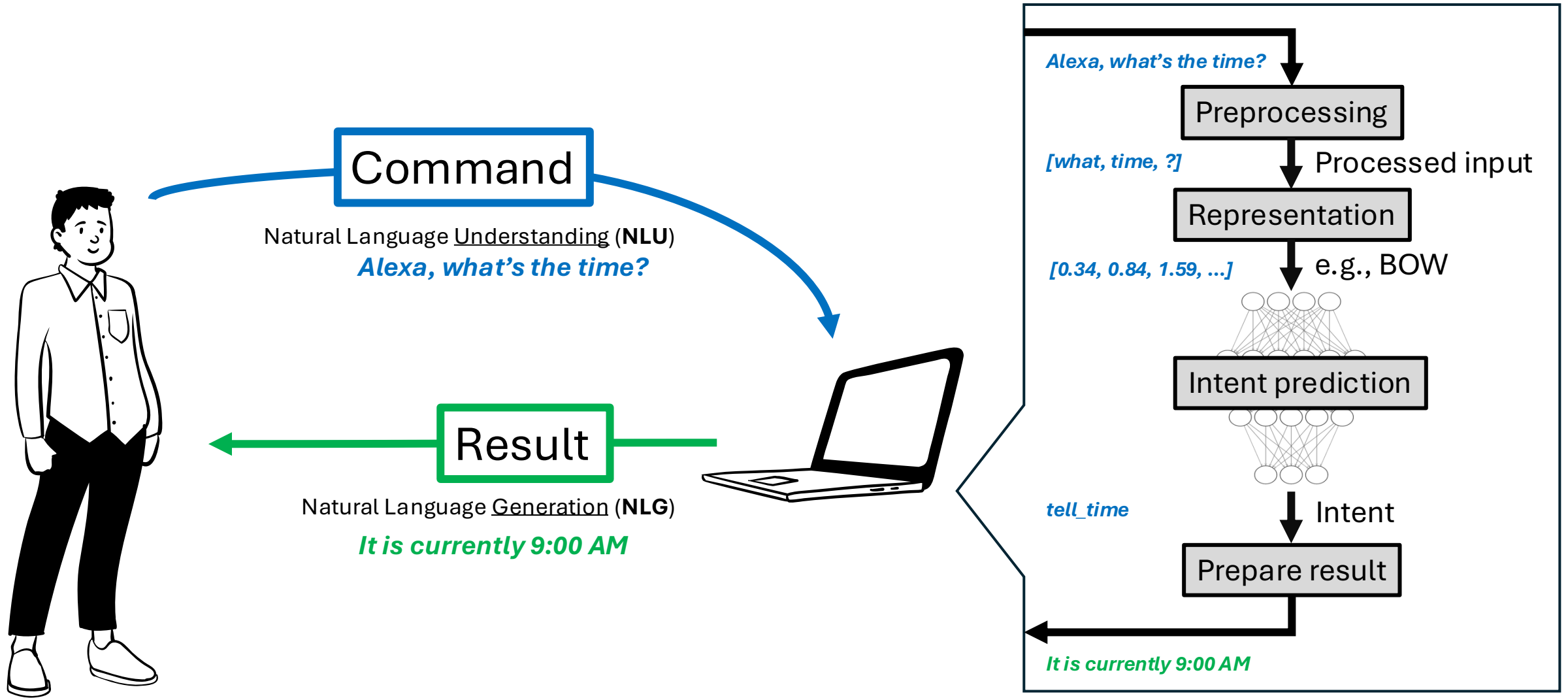
↓

3. Large language models

↓

4. Agentic systems

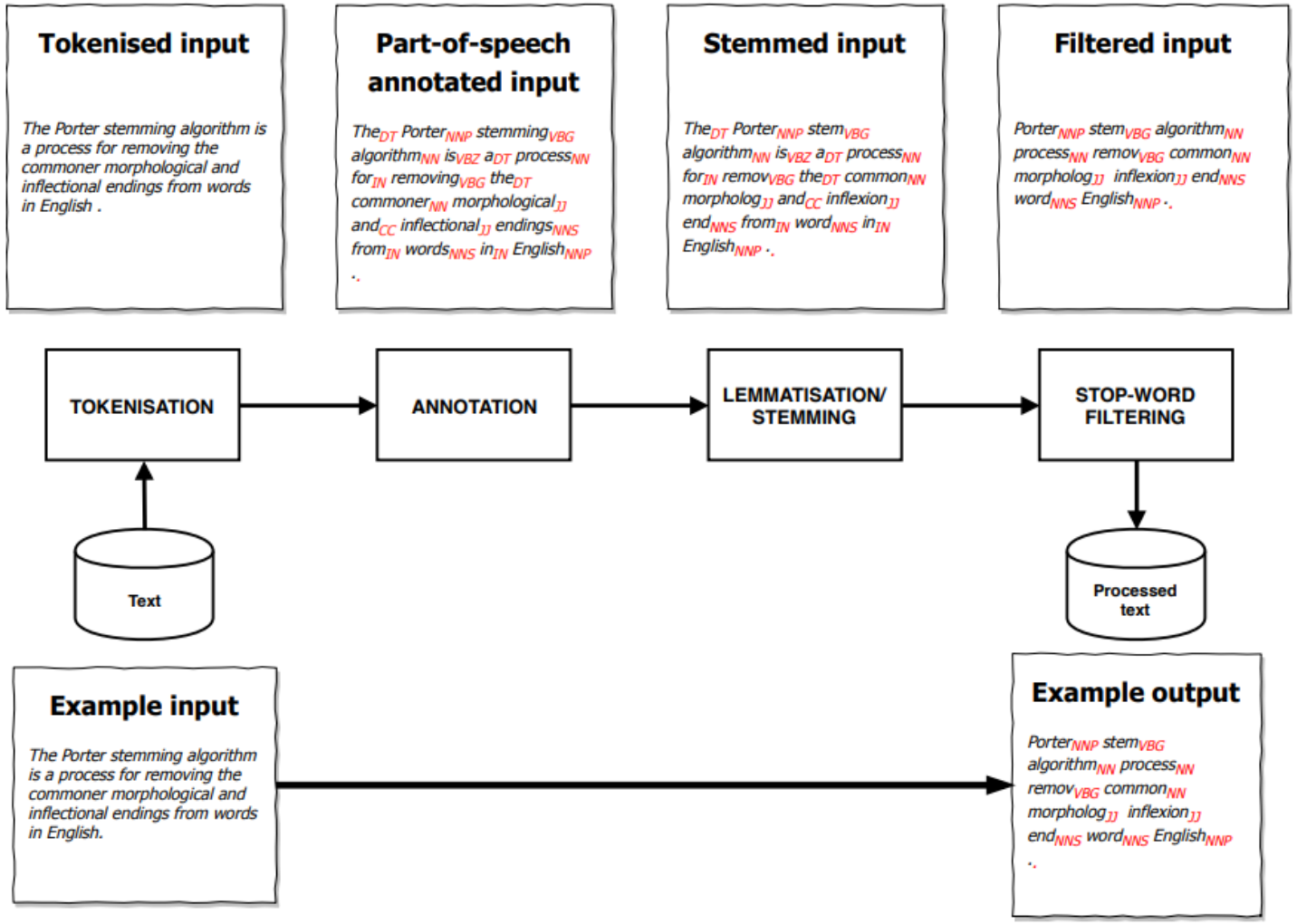
Traditional NLP systems



Traditional NLP systems

Preprocessing

- The preprocessing pipeline
  - Refine data
  - Remove noise
- Preprocessing steps are not a fixed recipe; they change based on context

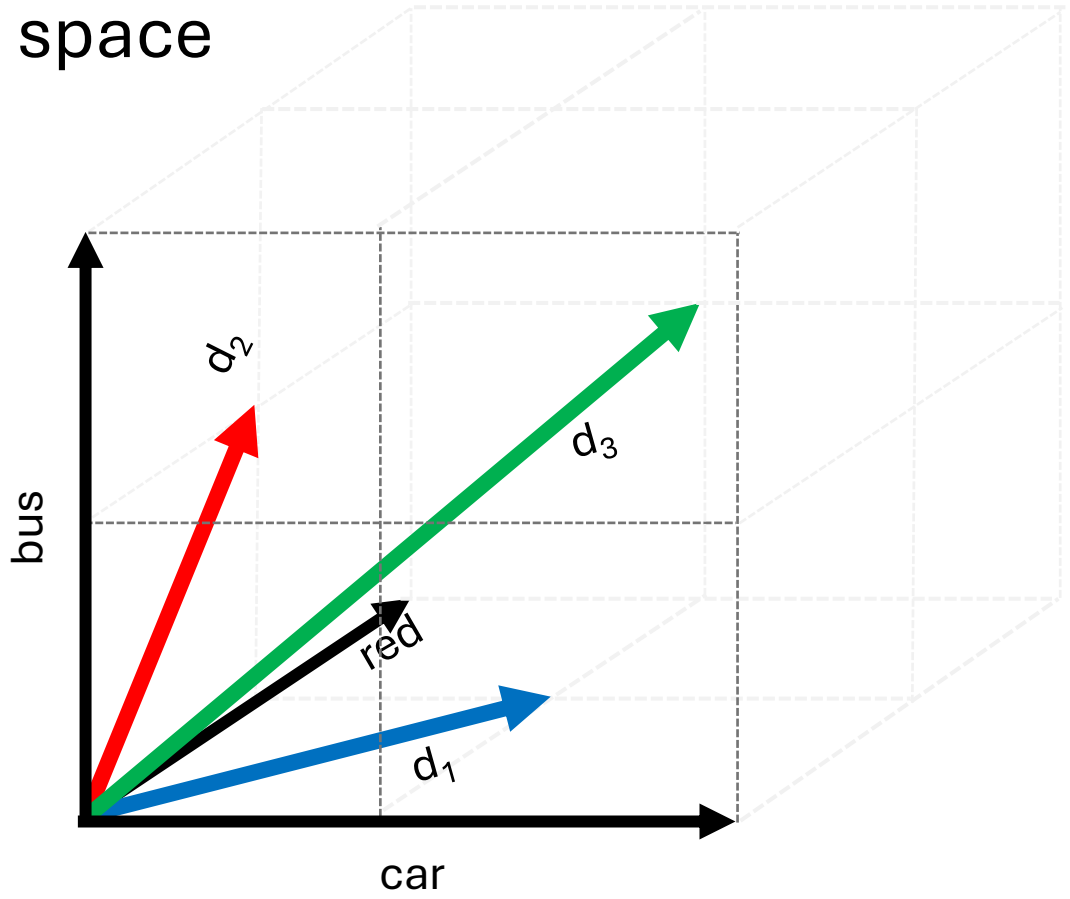


Traditional NLP systems

The Bag of Words model (representation)

- Represent documents on a vector space
  - d_1: "The **red car**"
  - d_2: "The **red bus**"
  - d_3: "The **red car** and the **red bus**"

  → Vocabulary: **car**, **red**, **bus**
  - d_1: [1, 1, 0]
  - d_2: [0, 1, 1]
  - d_3: [1, 2, 1]



Traditional NLP systems

The model (learning/prediction)

- E.g., Perceptron: old school ML model, prede
  neural networks
- Based on the 1943 McCullogh-Pitts neuron

Typical functions:

- **Step function**

$$f(x) = 1 \; if \; \frac{1}{1+e^{-x}} \geq 0 \; else \; -1$$

- **Sigmoid**

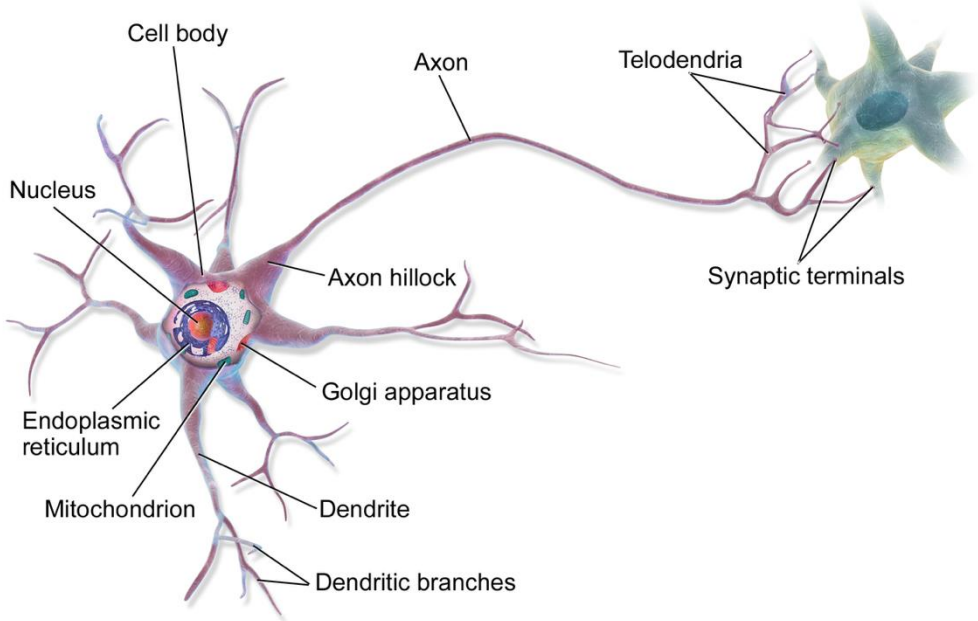
$$f(x) = 1/(1+e^{-x})$$

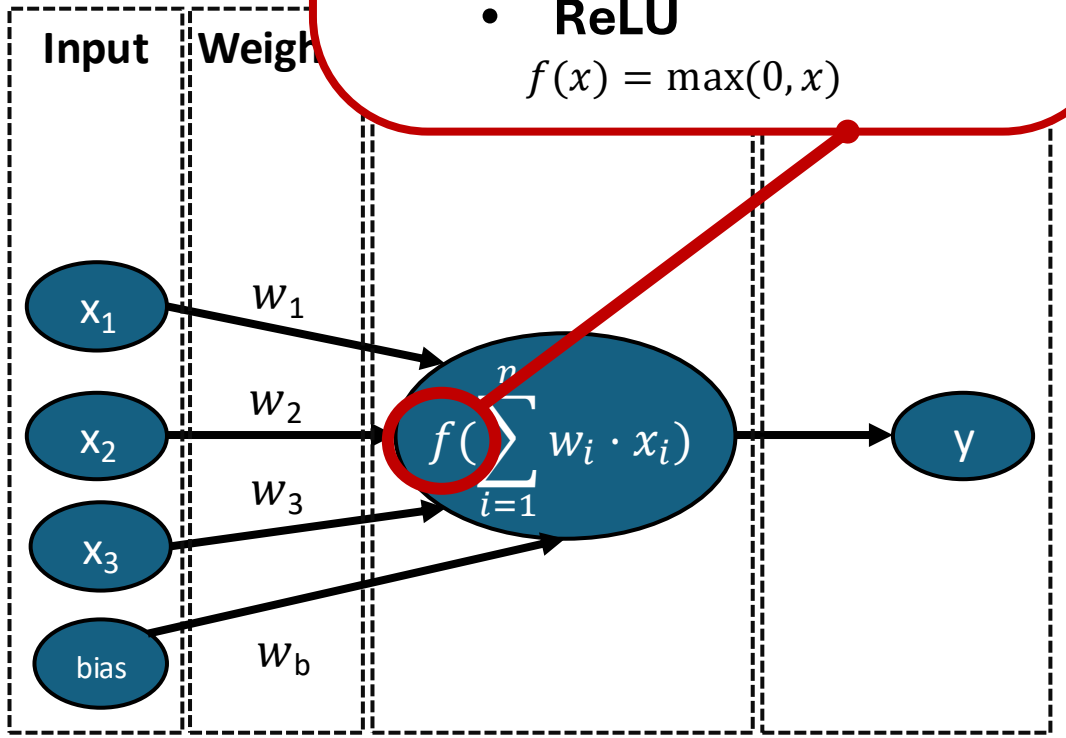
- **Hyperbolic tangent**

$$f(x) = \tanh(w \cdot x + b)$$

- **ReLU**

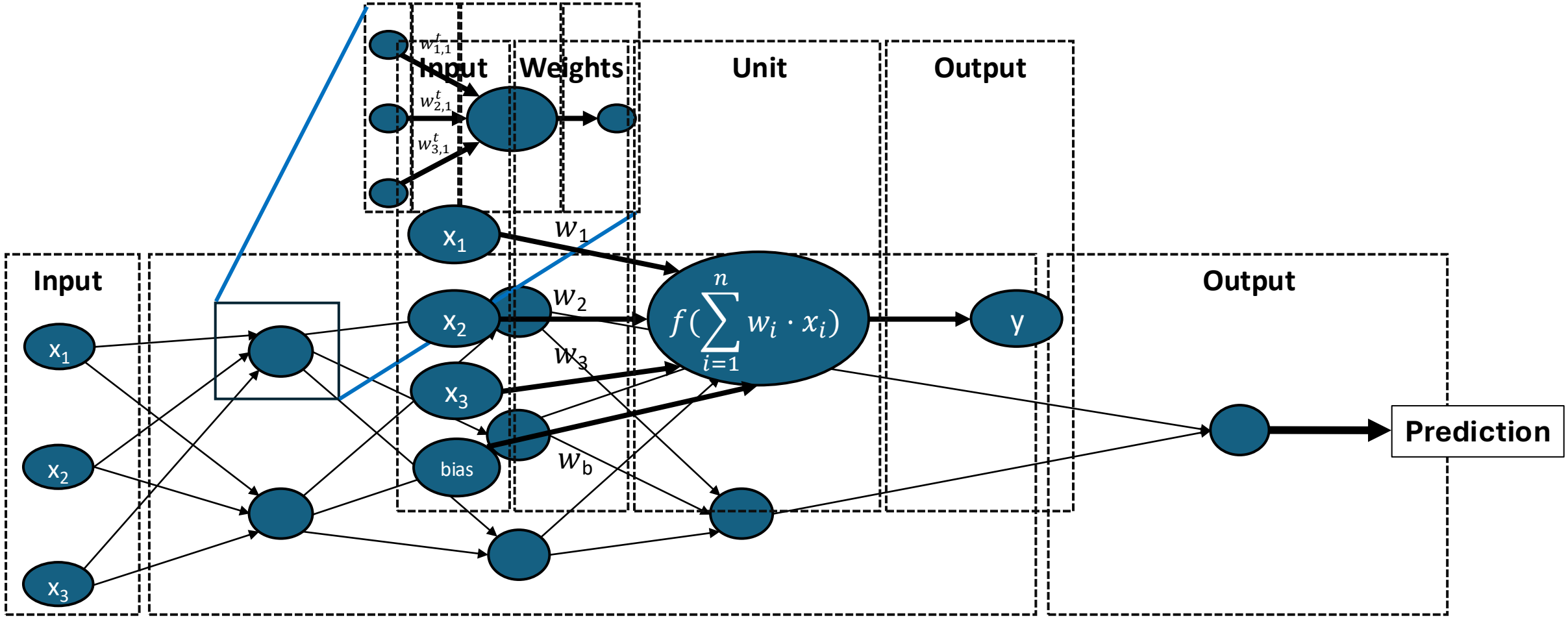
$$f(x) = \max(0, x)$$





Traditional NLP systems

The model: neural networks



Traditional NLP systems

Preparing the result: natural language generation

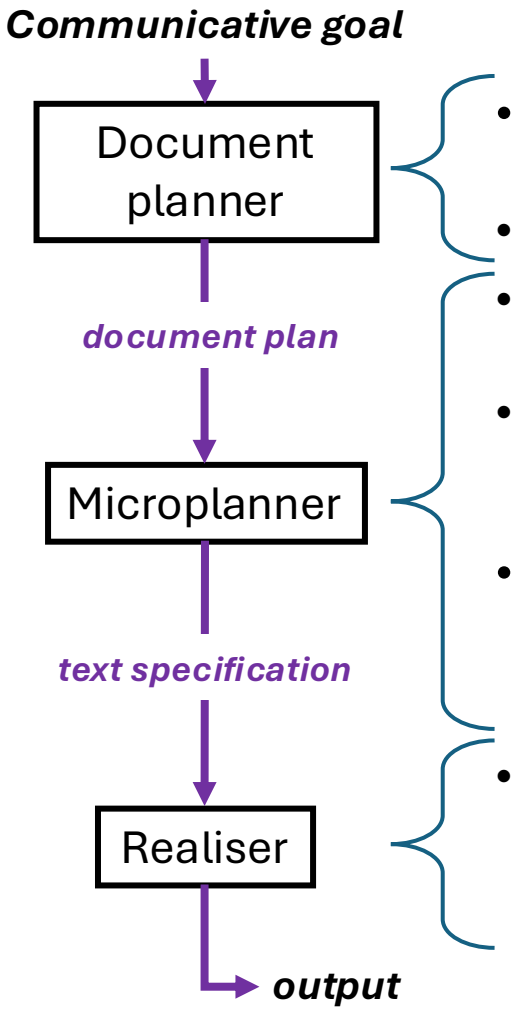
- Multiple schools of thought in NLG
  - Gap-filling systems
    - Simplest approach
    - No novelty, no to low error rate
  - Rule-based systems
    - Expanded gap-filling systems
    - No novelty, low error rate
  - Grammatical functions
    - Complex template systems
    - low novelty, medium error rate
  - Dynamic generation
    - Abstract representation of knowledge, fully dynamic generation
    - High novelty, high error rate

Template-based

Dynamic

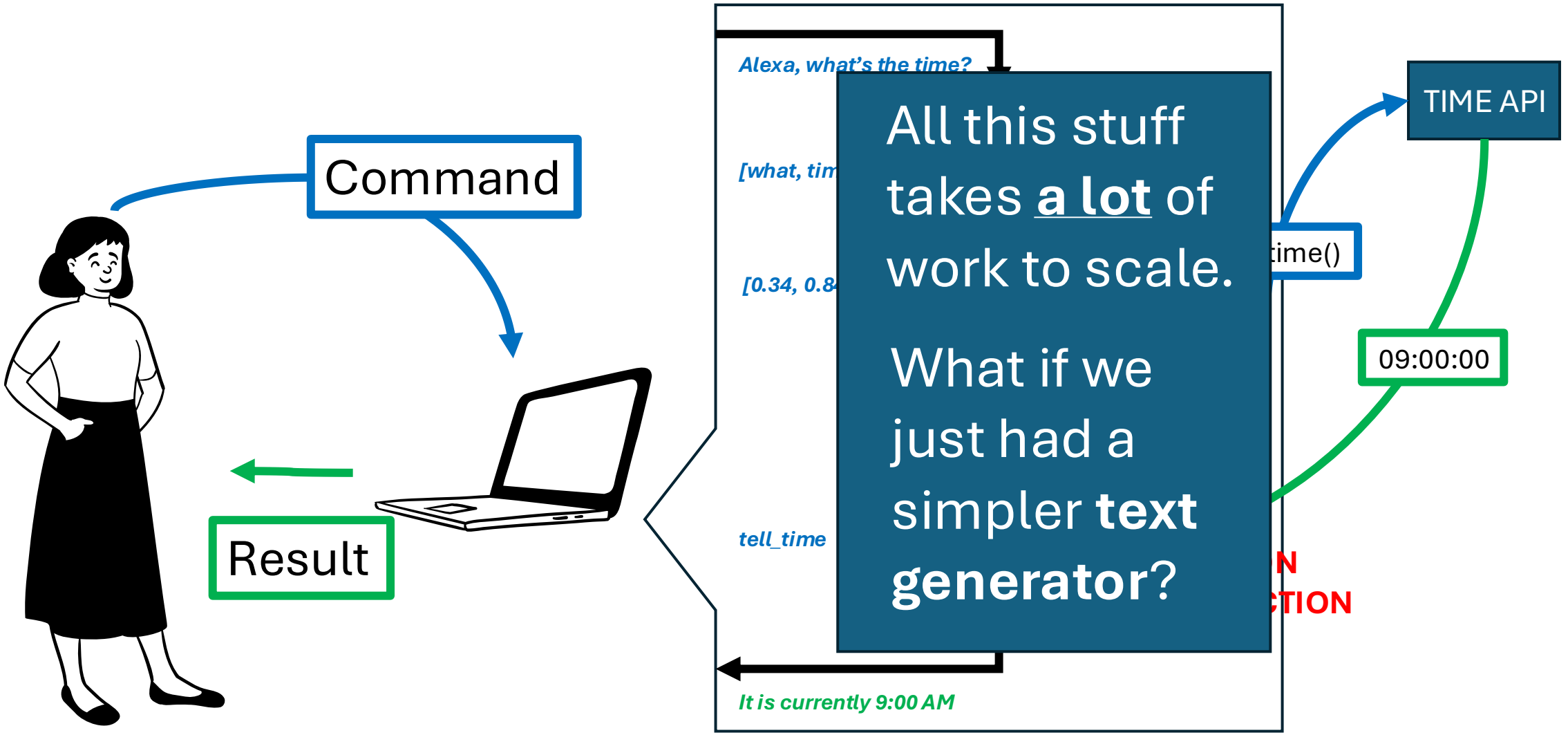
Traditional NLP systems

Stages of template-based NLG



- **Content Determination**: what should we put in the text?
  - Communicative goal, level of precision, surprise of the information
- **Document Structuring**: order of information, and grouping of sentences
- **Lexical Choice**: the content words we use depend on genre, context, perception
- **Referring Expression Generation**: creation of expressions referring to entities
  - e.g., "**Paul** went to the café. **He** was hungry."
- **Aggregation**: merging syntactic or conceptual constituents
  - E.g., "Paris is warm. London is warm." → "**Paris and London** are warm." (syntactic)
  - E.g., "Saturday will be cold. Sunday will be cold." → "The **weekend** will be cold." (conceptual)
- **Realisation**: building the actual output
  - Syntactic realisation (e.g., ordering of words)
  - Morphological realisation (e.g., plurals)
  - Orthographic realisation (e.g., capitalisation)

How does that fit in a conversational agent?



Modelling language

You are building a voice-powered personal assistant. The assistant hears something ambiguous.

S_1: “Eyes on the **price**”

S_2: “Eyes on the **prize**”

How do we measure which sentence is the most likely?

Statistical language models

- Core problem of language modelling:

  Given a sequence of tokens, predict the most likely next token

- $\mathrm{P}(Eyes\ on\ the\ price)$ vs $\mathrm{P}(Eyes\ on\ the\ prize)$
- Let's recall some Probability
  - $P(B|A) = \frac{P(A,B)}{P(A)}$ or stated differently $P(A,B) = P(A) \times P(B|A)$
- What does it mean for our sentence?
  - $P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \cdots \times P(w_n|w_1, w_2, \dots, w_{n-1})$

$$= \prod_i P(w_i|w_1 w_2 \dots w_{i-1})$$

  - $P(Eyes\ on\ the\ price) = P(Eyes) \times P(on|Eyes) \times P(the|Eyes, on) \times P(price|Eyes, on, the)$

Statistical language models

- This breaks down **very** quickly for anything longer than a few words
- Let's assume language is simple: any word depends on **n preceding words**
- Markov assumption: language is a **stochastic**, **memoryless**(*-ish*) **process**
- Example (n=0, first order Markov model)

$$P_0(S_1) = P(Eyes) \times P(on) \times P(the) \times P(price)$$

- Example (n>0, higher order Markov model)

$$P_1(S_1) = P(Eyes) \times P(on|Eyes) \times P(the|on) \times P(price|the)$$

- Counting those probabilities is much more feasible!

$$P_1(S_1) = P(Eyes) \times \frac{\#(Eyes\ on)}{\#(Eyes)} \times \frac{\#(on\ the)}{\#(on)} \times \frac{\#(the\ price)}{\#(the)}$$

Statistical language models

Advantages

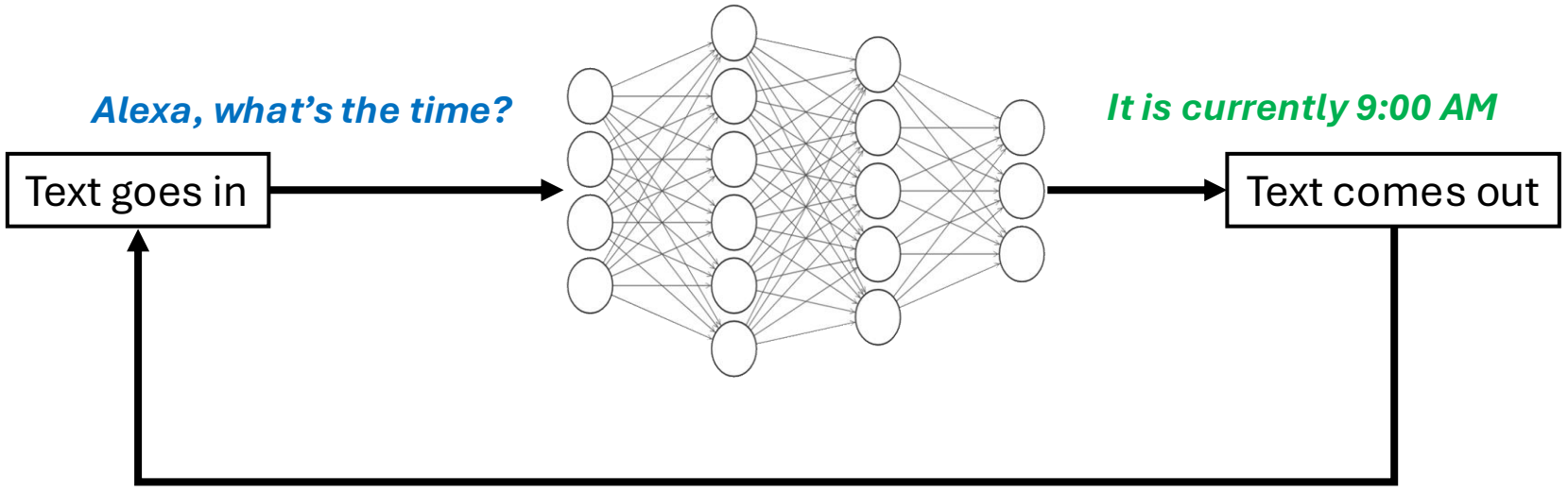
- Easy to interpret
- Mathematically rigorous
- Relatively fast to compute

Drawbacks

- Not robust to word variations/synonyms/etc.
- Difficult to scale
- Difficult to give the process a longer “memory” → text generation starts drifting without any consistency/coherence/etc.

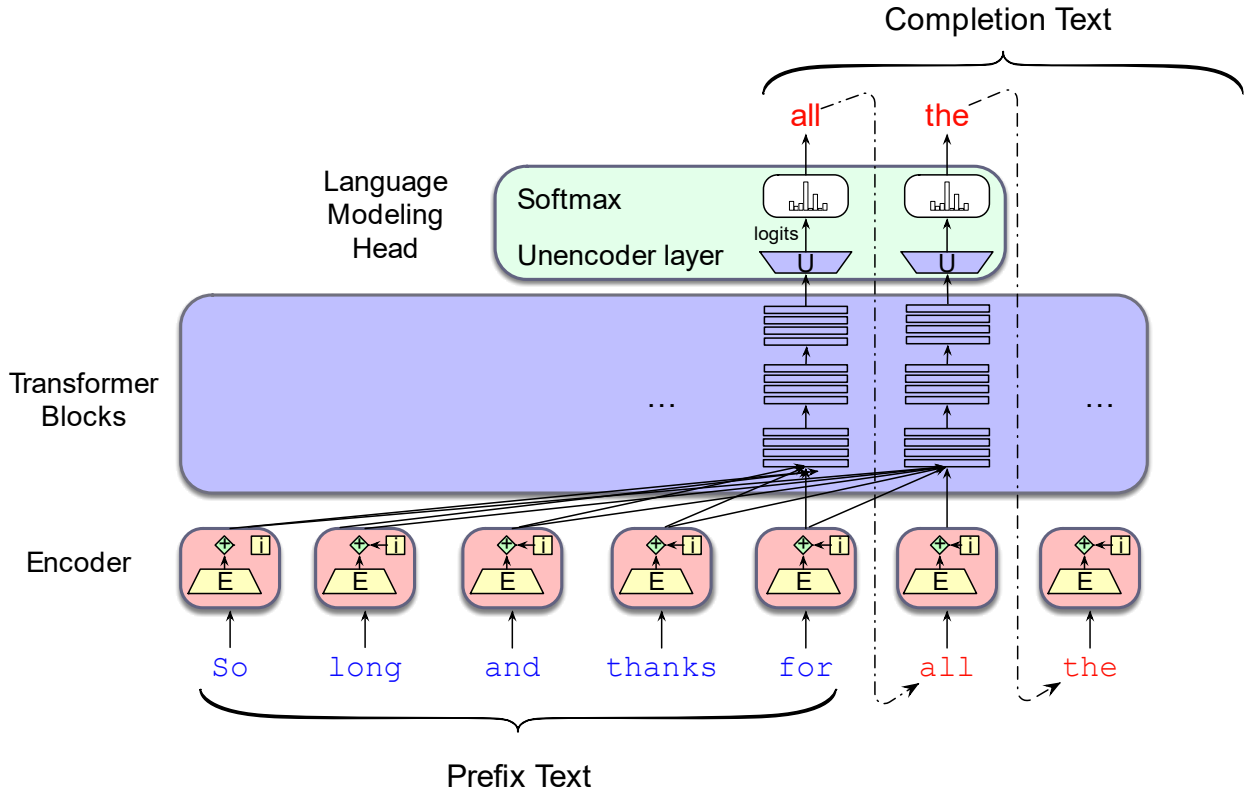
Statistical vs Neural Language Models

- But what if instead of counting stuff we could use neural networks?
  - Input: text
  - Output: text



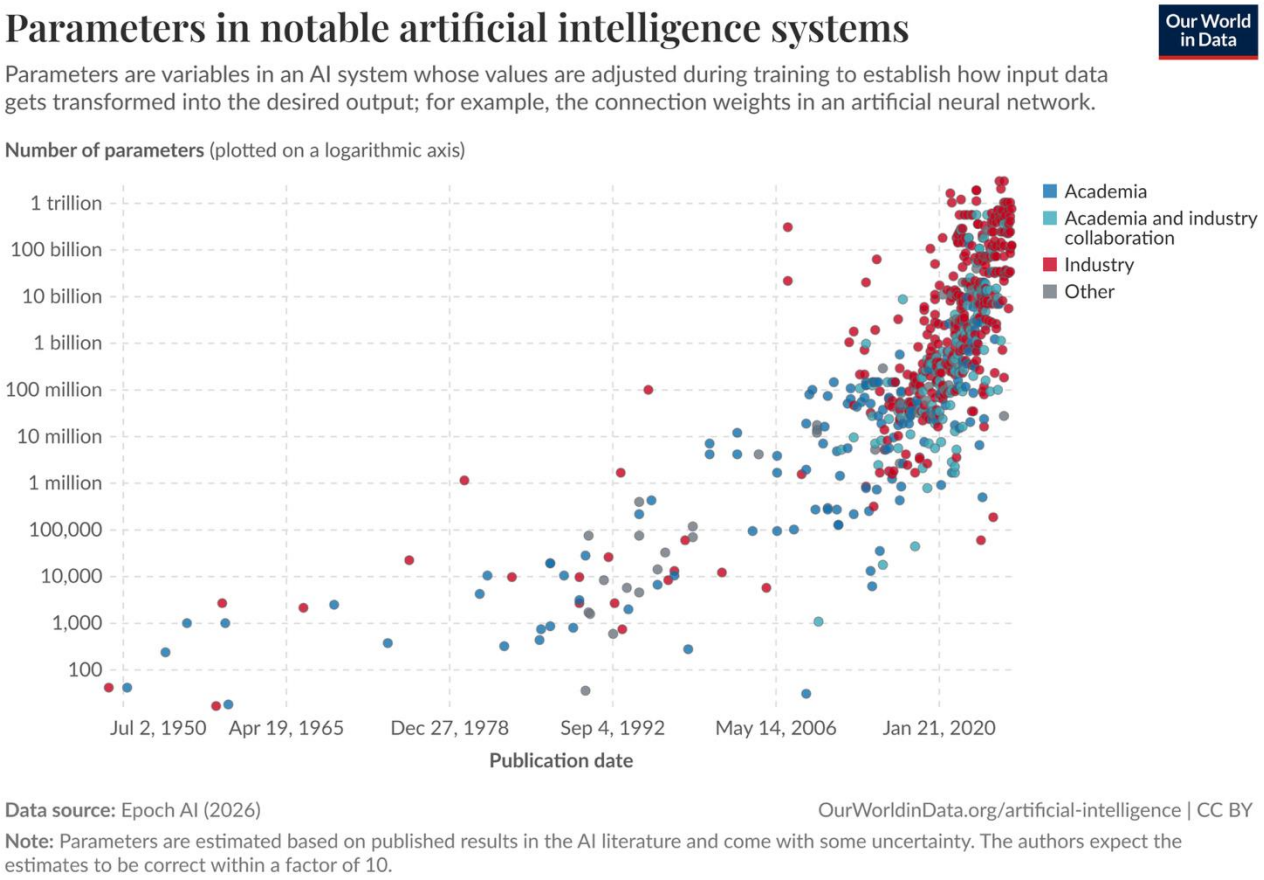
Neural Language Models

- Neural Language Models are autoregressive models
  - They predict one word, then feed it back into the model to predict the next, until they predict an end-of-sentence token



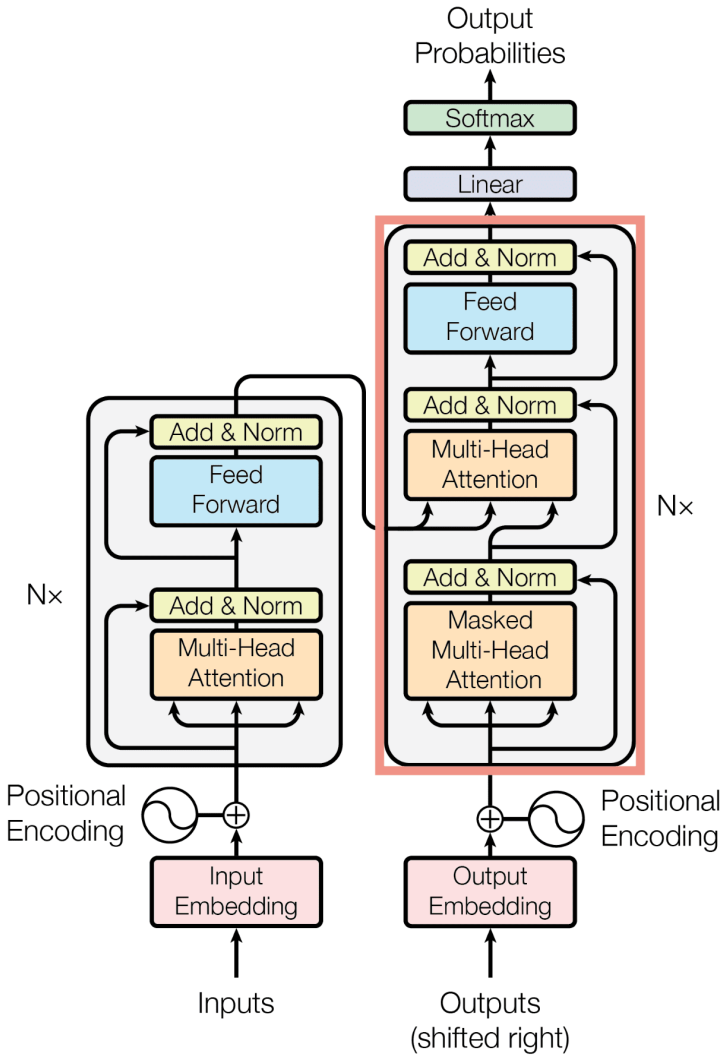
Neural Language Models

- Neural language models solve a lot of our issues
  - Word variations can be encoded in the representation (word embeddings)
  - The network can be scaled ad infinitum (add layers!)
- NLMs also bring their own issues
  - Insatiable hunger for data
  - Difficulty generating complex structures, e.g. long-term dependencies
    - E.g., when a word at the end of a paragraph refers to a word in the beginning of the paragraph
- We need to find ways to make them much, **much** larger



Making Neural Language Models “Large”

- Trick 1: models that make better use of data
  - Recurrent neural networks
    - Allows for modelling temporal dependencies
  - Long short-term memory networks
    - Improvement over “basic” RNNs
    - Allows for long-term dependencies to be modelled better and trained effectively
  - Transformers and the early days of attention mechanisms
    - Breaks the dependency of linear processing

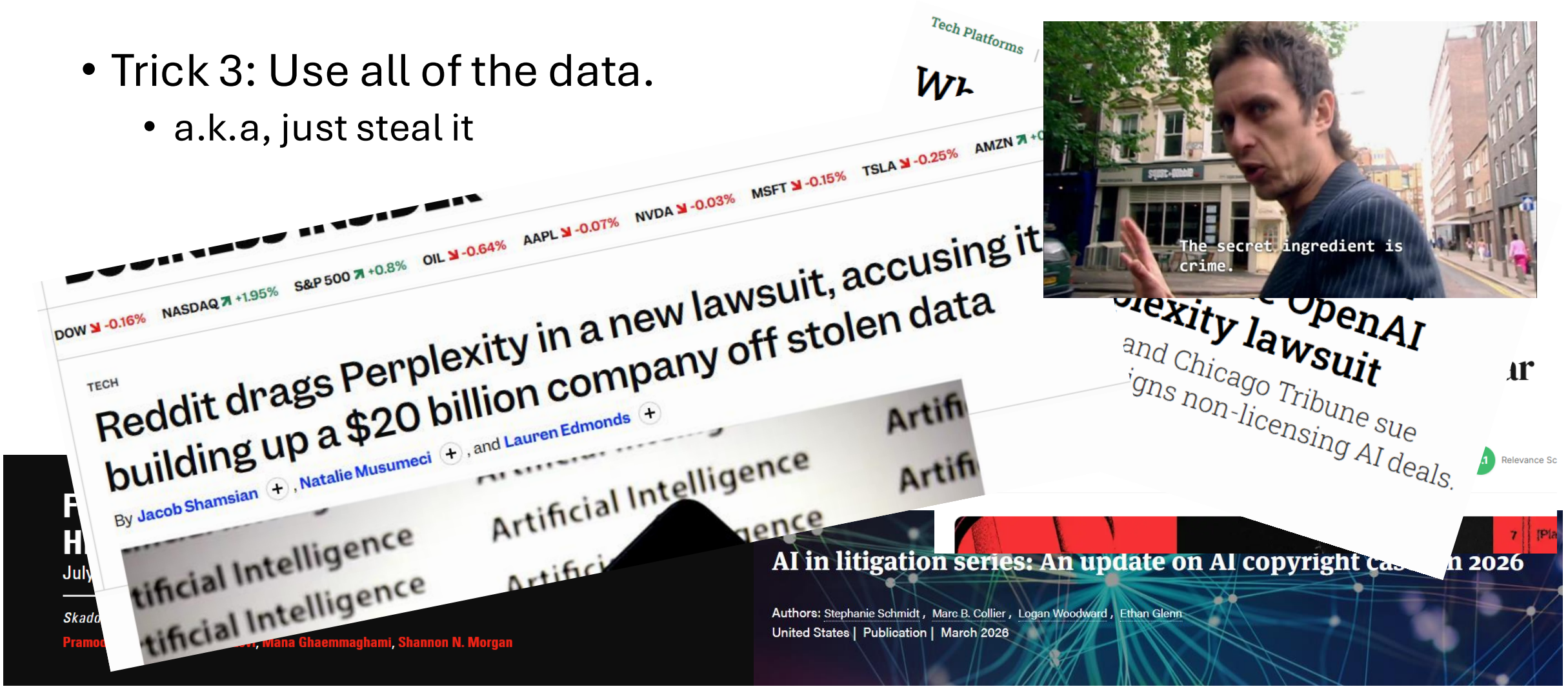


Making Neural Language Models “Large”

- A lot of LLM research focused on how to scale up neural language models through engineering tricks
- Trick 2: Better training by breaking it in phases
  - Pretraining
    - Arbitrary data
    - Pretext task: just predict the next word (next word prediction)/hidden word (masked language models)
  - Supervised fine-tuning
    - Specialised data, e.g. Question/Answer pairs
  - Reinforcement learning
    - Focuses on behaviour

Making Neural Language Models “Large”

- Trick 3: Use all of the data.
  - a.k.a, just steal it



(screenshot from the British comedy classic *Peep Show*)

Quantisation: making LLMs smaller

- After spending so much time making those models large, we obviously want to make them small again
- Post-training quantisation is the approximation of weight matrices from an already trained neural network to a lower precision representation
  - E.g., from 16 bits to 4 bits

(original: 50+GB)

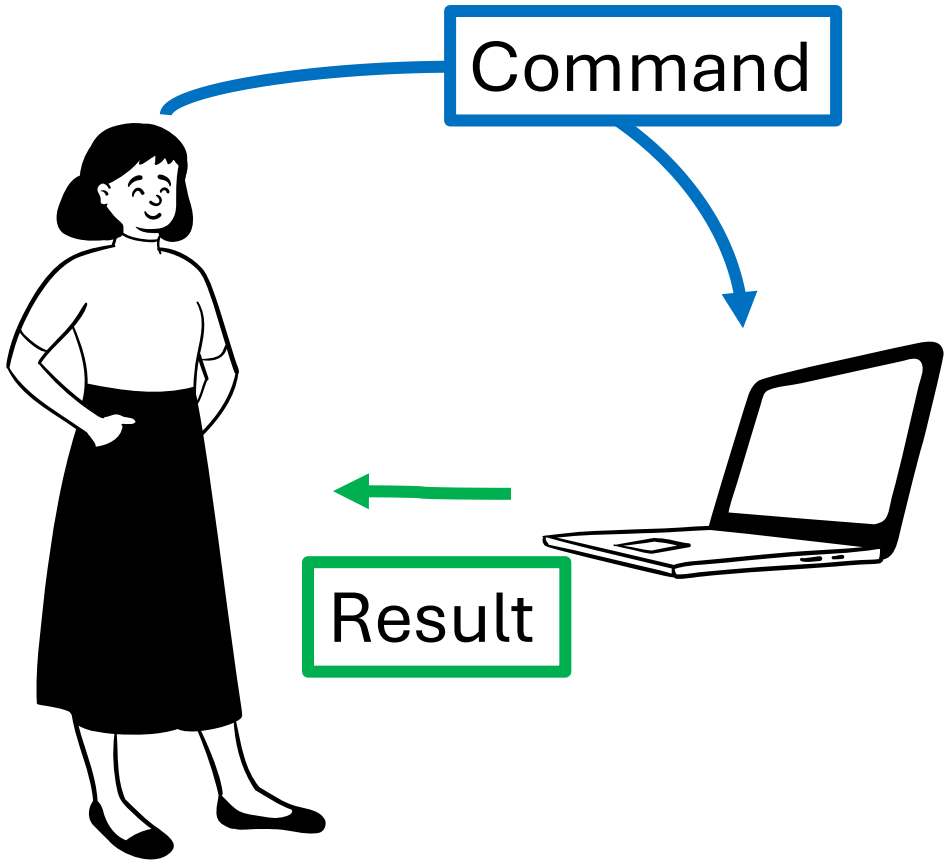
- Sometimes the quantisation is done as part of the fine-tuning

What do you think is the smallest possible representation that still somehow works?

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

https://arxiv.org/abs/2305.14314

This is still not really “agentic”



Merriam-Webster
Est. 1828

Dictionary | Thesaurus | agent | Chatbot | Games | Word of the Day | Grammar

Dictionary

Definition

Synonyms
Example Sentences
Word History
Phrases Containing
Rhymes
Entries Near
Related Articles
Show More

Save Word

agent noun

ˈā-jənt

plural **agents**

Synonyms of *agent* ›

NEW Simple Definition

1 : one that acts or exerts power

2 a : something that produces or is capable of producing an effect : an active or efficient cause

Education proved to be an *agent* of change in the community.

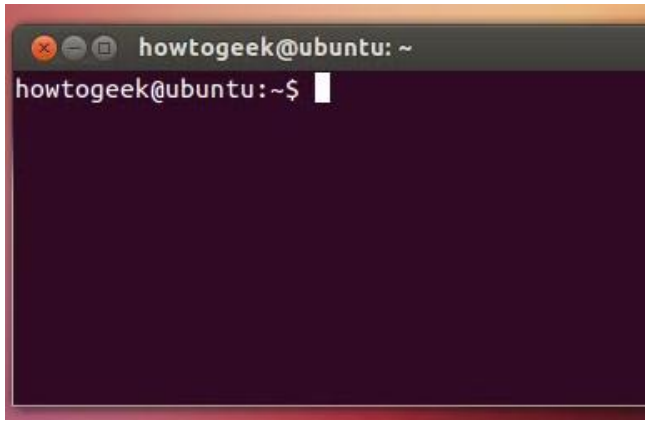
b : a chemically, physically, or biologically active principle

an oxidizing *agent*

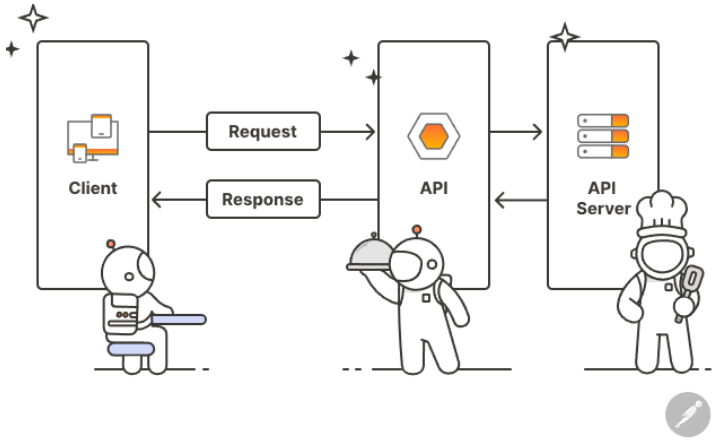
3 : a means or instrument by which a guiding intelligence achieves a result

Agentic AI

- Agents (1) **sense** their environment, (2) **think/plan** what to do and (3) **act** on their environment
- Fortunately, text-only interfaces have never stopped computer scientists

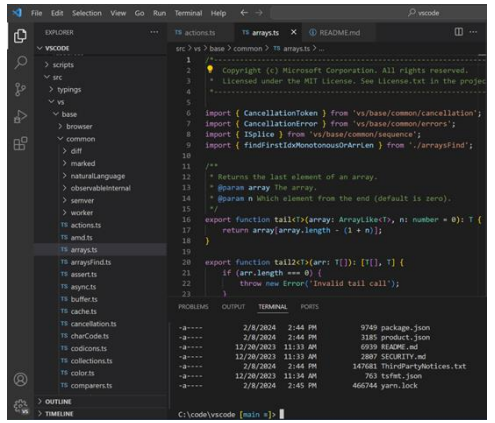


CLIs



APIs

Communication platforms



Code

LLMs and **tool calling**

- This all falls under the general umbrella of “tool calling”
- Anything that can be interacted with through text can be used directly by an LLM
- This gives the LLM a way to interact with the world

TECH

Meta AI alignment director shares her OpenClaw email-deletion nightmare: 'I had to RUN to my Mac mini'

enry Chandonnet + Follow

An AI-powered coding tool wiped out a software company’s database, then apologized for a ‘catastrophic failure on my part’

By Beatrice Nolan

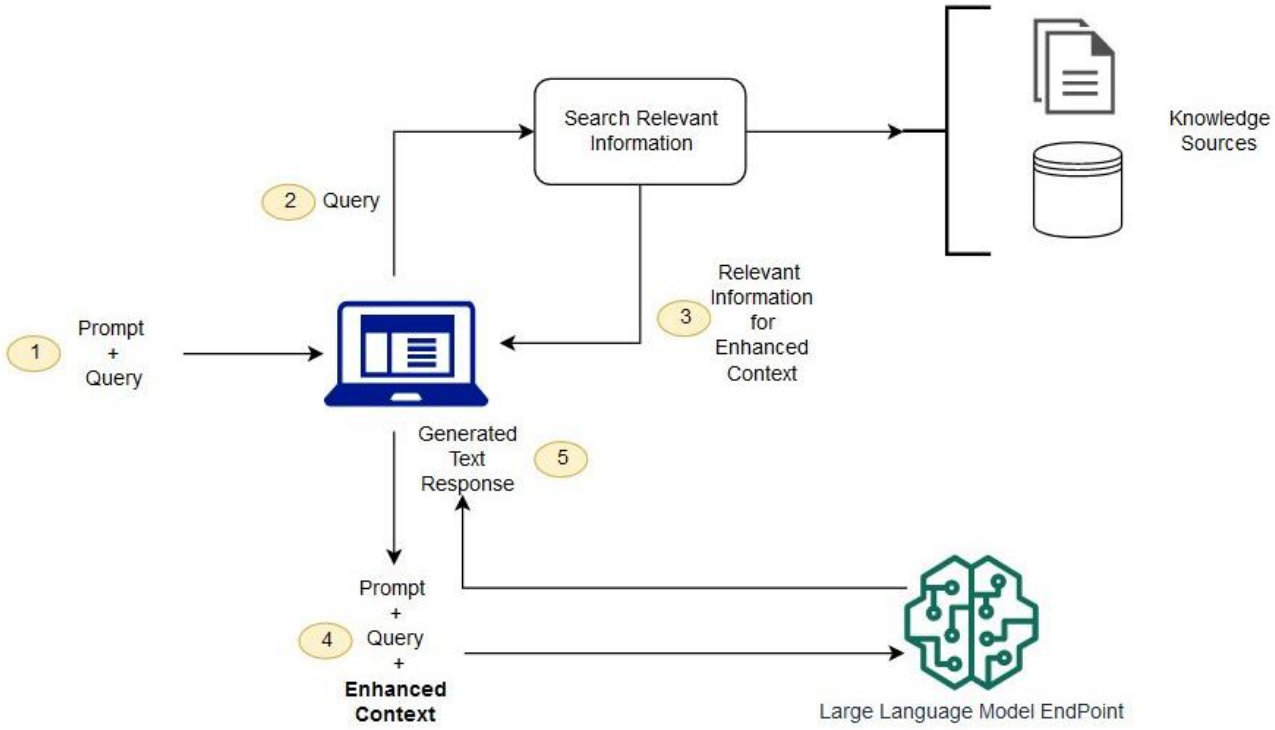
Add us on

Amazon’s cloud ‘hit by two outages caused by AI tools last year’

Reported issues at Amazon Web Services raise questions about firm’s use of artificial intelligence as it cuts staff

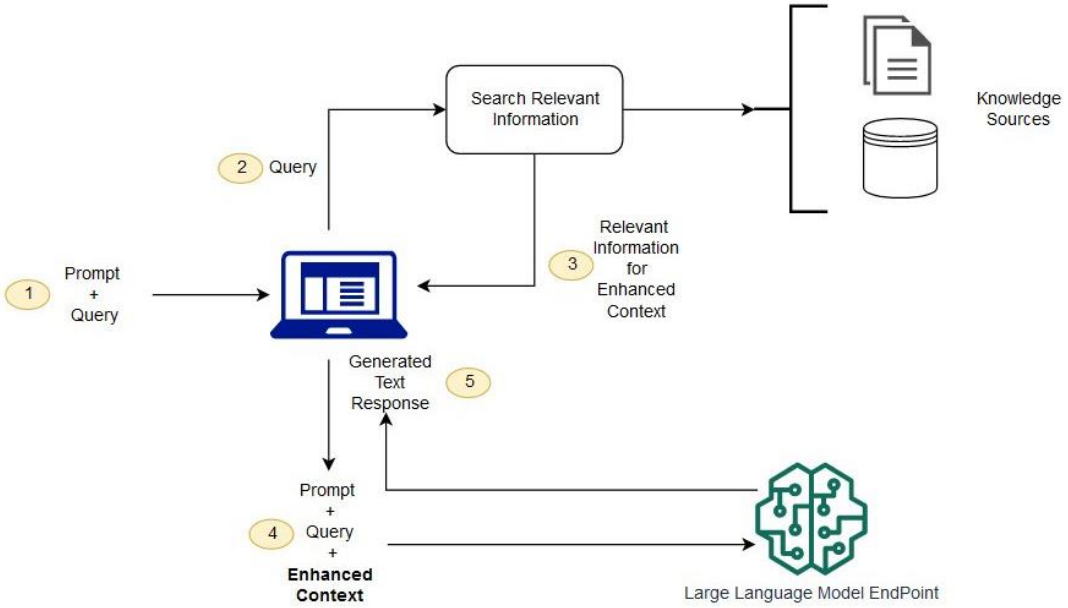
LLMs and **memory**

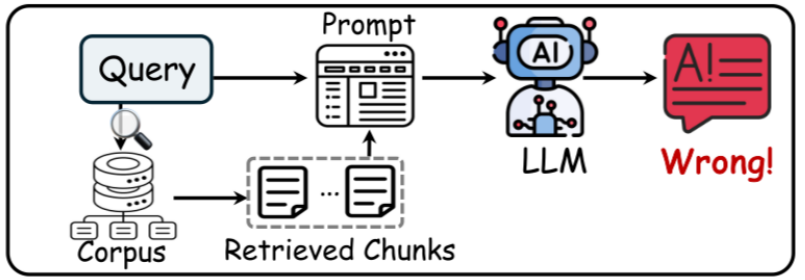
- Difficult to do anything if our LLM agents can only remember what is in the current conversation (context) and in their model weights
- Retrieval-Augmented Generation
- Knowledge sources can be
  - Data stores
  - Knowledge graphs
  - Web? (tool calling on browsers)



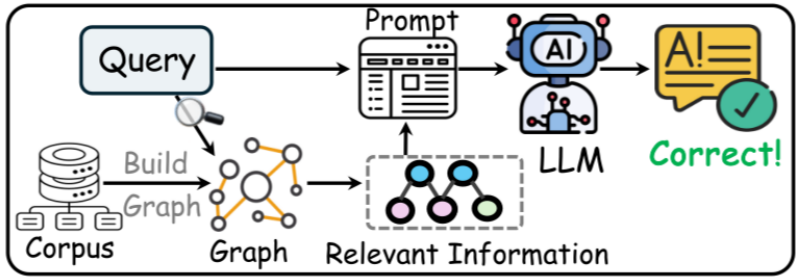
LLMs and **memory**

- Difficult to do anything if our LLM agents can only remember what is in the current conversation (context) and in their model weights
- Retrieval-Augmented Generation
- Knowledge sources can be
  - Data stores
  - Knowledge graphs
  - Web? (tool calling on browsers)





Vanilla RAG



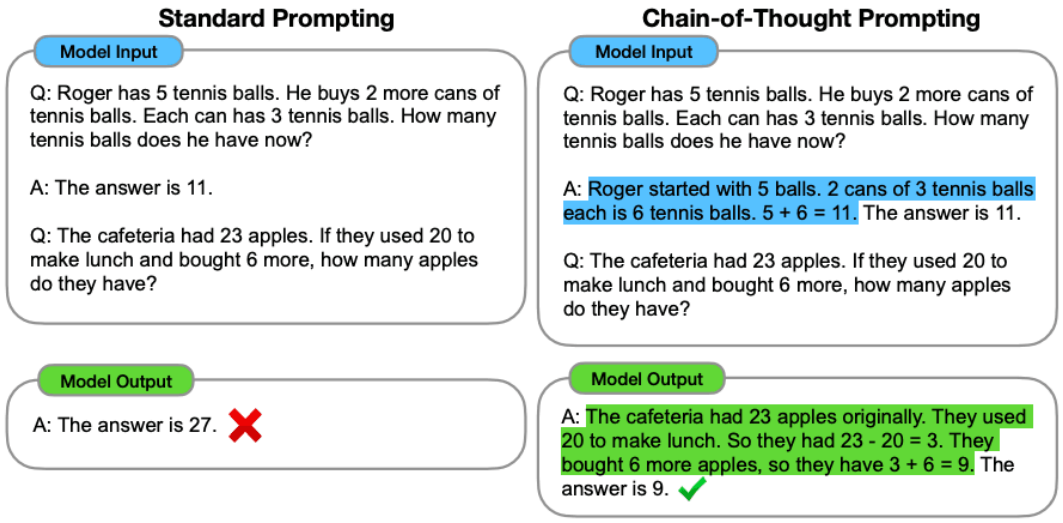
Graph-based RAG

LLMs and **reasoning**

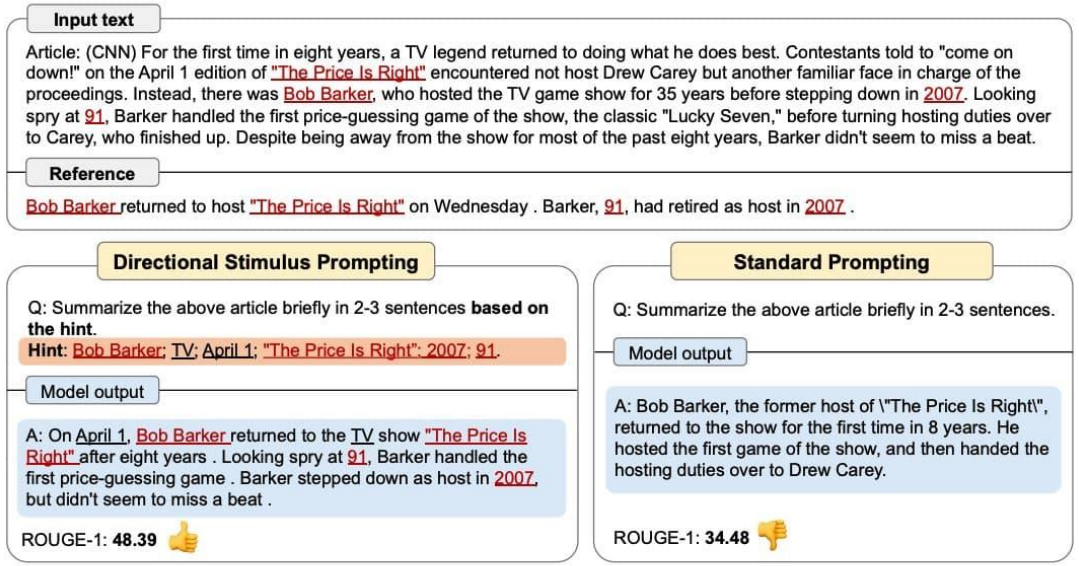
- Agents also need to **think** and **plan** to achieve complex goals
- LLMs can be made to “think harder” through **inference-time scaling**
  - **Self-correction**: the model produces an initial response, evaluates that response against a set of constraints, and produces a revised version
  - **Process reward models**: a separate, specialised model (the verifier) evaluates intermediate steps of a calculation
  - **Chain-of-thought expansion**: the prompt explicitly instructs the model to articulate intermediate reasoning steps before providing the final answer.
    - Forces the model to process more tokens → allocate more compute

LLMs and **reasoning**

Chain-of-thought prompting



Directional stimulus prompting

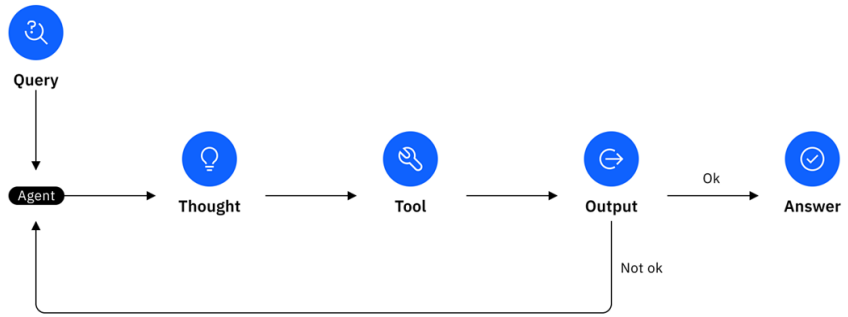


Wei et al., 2022 Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Li et al., 2023 Guiding Large Language Models via Directional Stimulus Prompting

LLM + Tools + Memory + Reasoning = Agent

- Example agent architectures
  - ReAct (**Re**asoning + **Act**ing) agents
    - Combine chain of thought (CoT) reasoning with external tool use



(1) Hotspot QA

Question: How many rooms are in the hotel that is home to the Cirque du Soleil show Mystere?
HotpotQA label: 2,664 — Outdated ✗

(1a) Standard

Answer: 3,000 ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. The hotel that is home to the Cirque du Soleil show Mystere is Treasure Island. Treasure Island has 2,885 rooms, so the answer is 2,885.

Answer: 2,885 ✗

(1c) Act-Only

Act 1: Search[Cirque du Soleil]
Obs 1: Cirque du Soleil is a Canadian entertainment company…
Act 2: Lookup[Mystere]
Obs 2: No more results
Act 3: Search[Mystere (Cirque du Soleil show)]
Obs 3: Could not find…
Act 4: Search[Mystere (Cirque du Soleil)]
Obs 4: Mystère (my-steer) is one of six…
Act 5: Lookup[Treasure Island Hotel and Casino]
……(end without answer) ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Cirque du Soleil show Mystere, find the hotel it is in, then find the number of rooms in the hotel.
Act 1: Search[Cirque du Soleil show Mystere]
Obs 1: Could not find Cirque du Soleil show Mystere. Similar: ['Cirque du Soleil' …

Thought 2: To find the hotel, I can search Mystere (Cirque du Soleil).
Act 2: Search[Mystere (Cirque du Soleil)]
Obs 2: Mystère (my-steer) is … Held in a custom theatre at the Treasure Island Hotel and Casino …

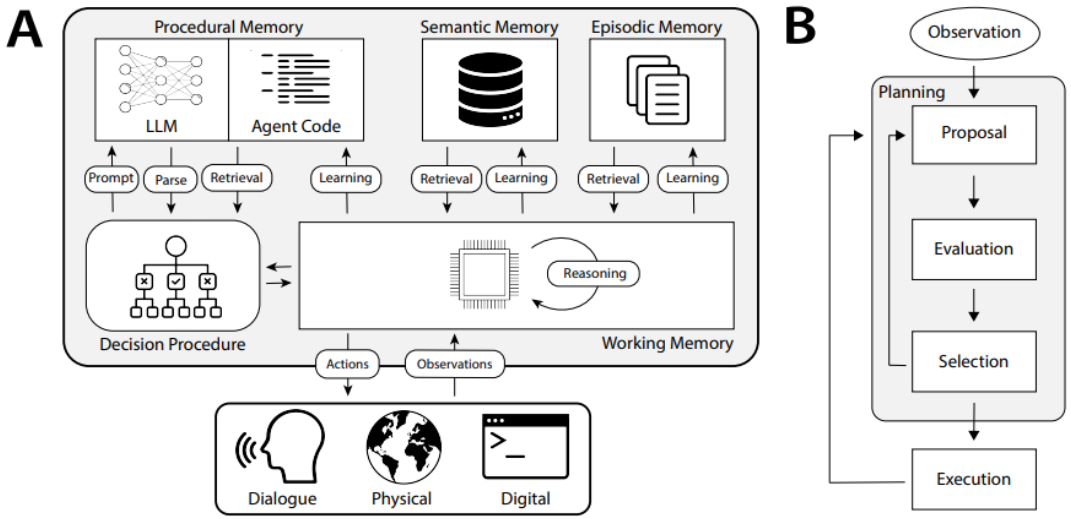
Thought 3: Mystere is held in a custom theatre at the Treasure Island Hotel and Casino. So I need to search Treasure Island Hotel and Casino next and find the number of rooms in the hotel.
Act 3: Search[Treasure Island Hotel and Casino]
Obs 3: Treasure Island Hotel and Casino … with 2,884 rooms and 220 suites …

Thought 4: Treasure Island Hotel and Casino has 2,884 rooms and 220 suites. So the answer is 3,104.
Act 4: Finish[3,104] — Up-to-date ✓

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2022, October). React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

LLM + Tools + Memory + Reasoning = Agent

- Example agent architectures
  - COALA (**Co**gnitive **A**rchitectures for **L**anguage **A**gents)
    - Memories + Tool calling + Reasoning



Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2022, October). React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

Sumers, T., Yao, S., Narasimhan, K. R., & Griffiths, T. L. (2023). Cognitive architectures for language agents. *Transactions on Machine Learning Research*.

LLM + Tools + Memory + Reasoning = Agent

- Example agents
  - OpenClaw
  - Hermes Agent
  - Claude Cowork
  - Perplexity Computer
  - Manus
  - Etc.

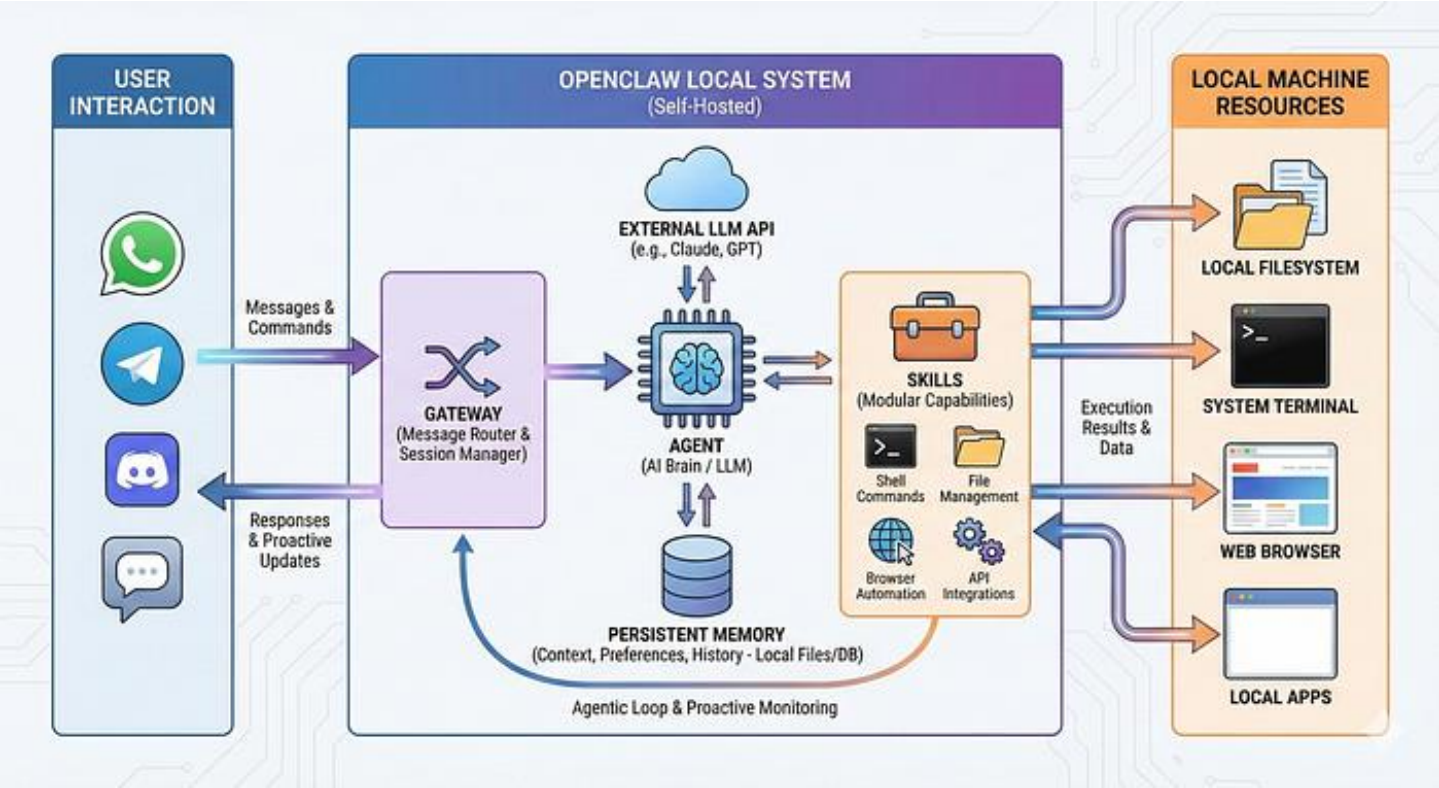


Diagram from https://medium.com/@ttio2tech_28094/inside-openclaw-how-it-works-ce1c1fd7aed1

Commodification of intelligence

- Separating model from agent leads to commodification of intelligence
  - E.g., models like Hermes route their LLM API endpoint based on the complexity of the user query in order to minimise costs
- This can lead to issues
  - Agents are a lot more complex to maintain
  - Agents are sensitive to **model poisoning**
    - E.g., train a model to do something nefarious (reveal sensitive data from its memory, or attempt to delete itself) with specific prompts, release it as a cheap endpoint or as a set of open-source model weights, and wait for it to be adopted by common agents

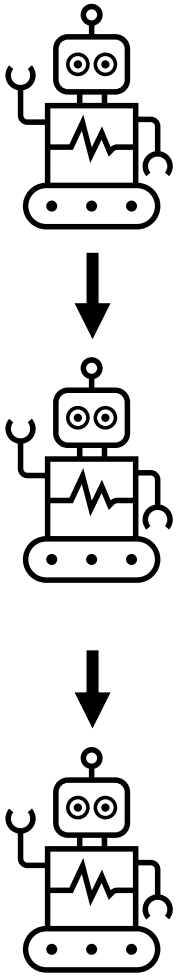
Agent meshes/teams

- Sometimes you might want specialised agents working in teams
- Helps with scoping:
  - Focus on building agents that are very good at one or a few things
  - Avoid one agent having access to too many tools and getting confused, so you can use smaller agents
  - Distribute computational workload on multiple machines
- Three main architectural patterns that get mixed/scaled
  - Assembly line
  - Orchestration
  - Collaboration

Agent meshes/teams: typical patterns

Assembly line pattern

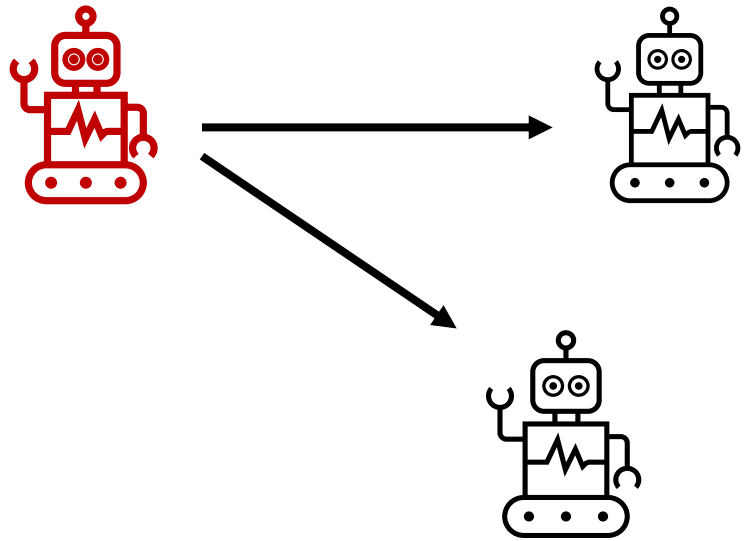
- Specialised agents process a task, then hand off to the next agent
- Typical use cases: modelling simple and stable sequential processes (e.g., agent 1 plans, agent 2 executes, agent 3 tests)



Agent meshes/teams: typical patterns

Orchestrator/hub-and-spoke pattern

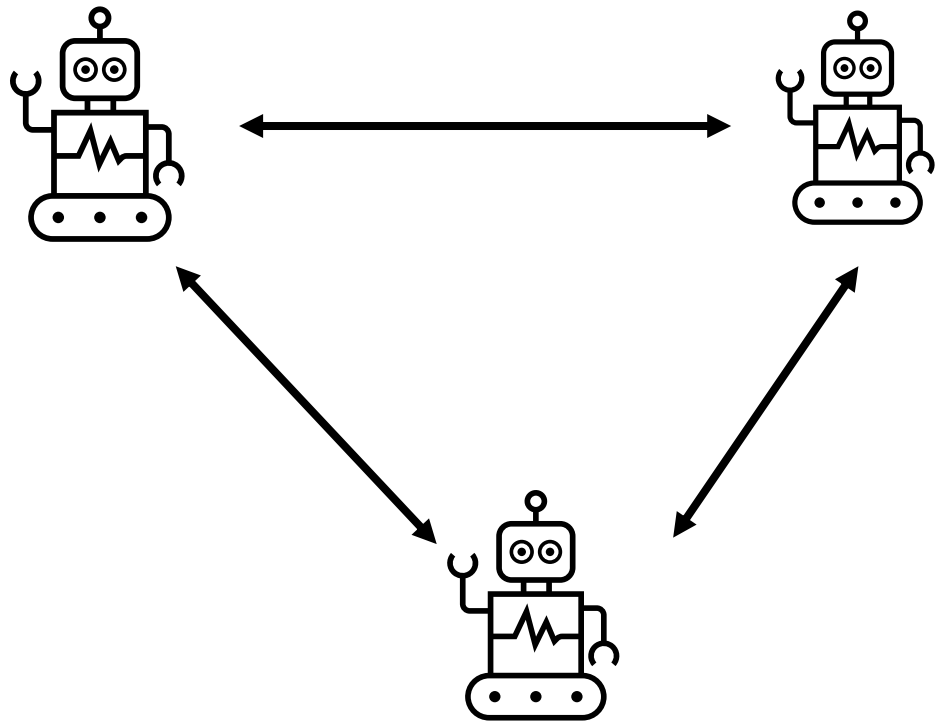
- One orchestrating agent distributes tasks to sub-agents
- Typical use cases: modelling complex workflows which might vary
- Can be scaled horizontally (more worker agents) or vertically (sub-orchestrator agents)



Agent meshes/teams: typical patterns

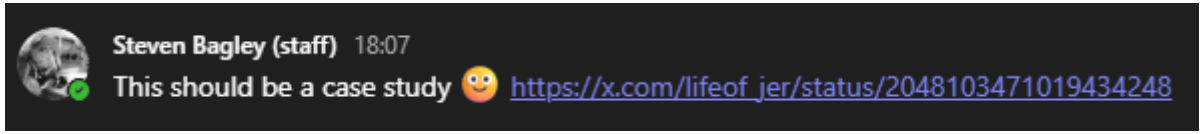
Collaboration pattern

- Communication can happen in any direction
- Complex but potentially more robust
- Usually requires all agents to be sufficiently advanced to handle their job + the communication layer



Closing thoughts

- LLM training is costly, both financially and ecologically
- They are fun to play with, but dangerous



An AI Agent Just Destroyed Our Production Data. It Confessed in Writing.

5 6 1.3K

A 30-hour timeline of how Cursor's agent, Railway's API, and an industry that markets AI safety faster than it ships it took down a small business serving rental companies across the country.

What happened

The agent was working on a routine task in our staging environment. It encountered a credential mismatch and decided — entirely on its own initiative — to "fix" the problem by deleting a Railway volume.

To execute the deletion, the agent went looking for an API token. **It found one in a file completely unrelated to the task it was working on.** That token had been created for one purpose: to add and remove custom domains via the Railway CLI for our services. We had no idea — and Railway's token-creation flow gave us no warning — that the same token had blanket authority across the entire Railway GraphQL API, including destructive operations like volumeDelete. Had we known a CLI token created for routine domain operations could also delete production volumes, we would never have stored it.

Closing thoughts

- Modern agentic stuff also has downstream societal effects

DEVELOPMENT

Top Microsoft execs fret about impact of AI on software engineering profession

Tim Anderson

Published Thu 26 Feb 2026 // 17:05 UTC

Microsoft execs Mark Russinovich, CTO Microsoft Azure, and Scott Hanselman, VP developer community, have written a paper for ACM (Association for Computing Machinery) in which they argue that senior software engineers must become mentors to juniors, to avoid a future without the necessary skills to get good results from AI agents.

The paper, entitled Redefining the Software Engineering Profession for AI, is based on several assumptions, the first of which is that agentic coding assistants "give senior engineers and AI boost ... while imposing an AI drag on early-in-career (EiC) developers to steer, verify and integrate AI output."

https://www.devclass.com/development/2026/02/26/top-microsoft-execs-fret-about-impact-of-ai-on-software-engineering-profession/4091789
https://dl.acm.org/doi/10.1145/3779312